marketpower2: An Interbank model with change of lender mechanism

Héctor Castillo Andreu

May 2026

1 Auxiliary files

- `pyproject.toml`: UV environment configuration and also list of the necessary python packages

2 Interbank model

- `interbank.py`: use to execute standalone the Interbank simulation.
  - It accepts command line options. For instance:

    ```
    interbank.py --log DEBUG --n 150 --t 2000
    interbank.py --save results.gdt p=0.4 param=X
    ```

  - When it is used as a package, the sequence should be:

    ```
    import interbank
    model = interbank.Model()
    model.config.configure(param=x)
    model.forward()
    ```

- Basic options:

    ```
    # To list all options:
    interbank.py --help

    # To run a simulation based on exp_runner:
    uv run experimments\exp_min_p_0_1.py --do
    # Same but using only surviving banks to
    # determine how last the system to die:
    python -m experiments.exp_surviving_4.py --do
    ```

- `interbank_lenderchange.py`: It contains the algorithm that control the change of lender in the model.
- `interbank_statistics.py`: Generates the stats of the model.

- `interbank_log.py`: Generates the logging.
- `interbank_testclass.py`: It contains the algorithm that control the change of lender in the model.
- `interbank_web.py`: Minimal web server for interbank using Flask.
- `exp_runner.py`: A prototype for executing experiments with different parameters and using MonteCarlo (using concurrent.futures to allow multiple threads).
- `exp_runner_surviving.py`: A derivation of the former prototype using ray library to execute in a cluster.
- `experiments/`: directory with all the experiments conducted. The results of that executions are stored in a folder determined inside each experiment.
- `doc/algorithm.pdf`: the PDF schema of the algorithm used in the model to propagate shocks and to balance sheets.
- `experiments/`: directory with all the experiments conducted. The results of that executions are stored in a folder determined inside each experiment.
- `output/`: directory with the output files (default one, you can change it using the '–output' option).

3 Basic usage of the model

- `interbank.py seed=1234 T=500 --p 0.2`: Execute the model with $T = 500$ and *LenderChange* algorithm of $ShockedMarket3$ with an Erdös-Réni with probability of attachment $p_a = 0.2$ and using a seed for generating random values of 1234 (same results if you generate again with other equal parameters and repeat this integer number for seed).
- `interbank.py --save result --output_format csv`: Save the results in `result.csv` in CSV. With `--log DEBUG --logfile result.txt` will save also the log in a file.
- `interbank.py --normalize_ir`: Execute the model normalization the interest rate to a range [0..1].
- `interbank.py --robust_ir`: Execute the model with a similar to a Robust Scaler algorithm normalization.

  In our implementation, robust_ir is a quantile-based min–max normalization applied per time step, not a true RobustScaler. Here is the formulation:

  1. Let determine first the set of interest rates at time t:

$$\mathcal{I}_t = \{r_i \,|\, r_i \in \mathbb{R},\ i = 1, \ldots, N_t\}$$

2. We compute the low and high quantiles of this set ($p_{\text{low}} = 5\%$ and $p_{\text{high}} = 95\%$ by default):

$$q_{\text{low}} = Q_{p_{\text{low}}}(\mathcal{I}_t), \qquad q_{\text{high}} = Q_{p_{\text{high}}}(\mathcal{I}_t).$$

3. We clip the outliers:

$$\tilde{r}_i = \max(q_{\text{low}}, \min(q_{\text{high}}, r_i))$$

4. And min-max normalize into $[0, R]$ (with $R = 1.0$ by default):

$$r_i^{(\text{robust})} = \begin{cases} R \dfrac{\tilde{r}_i - q_{\text{low}}}{q_{\text{high}} - q_{\text{low}}}, & q_{\text{high}} > q_{\text{low}} \\ 0, & q_{\text{high}} = q_{\text{low}} \end{cases}$$

4 Web server

A Web interface is available to run the model and visualize the results in charts. It is implemented with Flask and can be launched with `interbank.py --web` option. With `--web_mode dashboard --web_port 8080` we obtain the same as previous command but explicitly setting the mode and port. Available modes are:

- `simulate`: single execution UI.
- `multiple`: parameter multiple-run UI.
- `dashboard`: combined UI with tabs, includes charts for simulate/multiple.

In dashboard, chart metric visibility is selectable with checkboxes. Metric selections are persisted in browser 'localStorage'. Backend routes are:

- `POST /api/simulate` for single runs.
- `POST /api/multiple` for multiple parameter runs.

The dashboard template file is `templates/template_simulation.html`.

5 Statistics

Different statistics can be obtained after running the model, either in **csv** output, or in **gdt** (Gretl format). This statistics collect data in each time for the average or individually, depending on the usage. Possible statistics obtained from the model are:

- **asset_i**: Assets of the lender of this bank ($D + E$)
- **asset_j**: Assets of the borrowers of this bank ($D + E$)

- **bad_debt**: Sum of the bad debt
- **bankruptcies**: Number of banks that failed in this step
- **capacity**: Lender capacity $(1 - \frac{E}{E_{\max}})$ of the bank
- **communities**: Subsets of nodes with higher internal edge density than connections to the rest of the graph
- **communities_not_alone**: Number of **communities** that are not formed by only one node
- **d1,d2**: Sum of demand of loan of borrowers not satisfied by their own capital after shock1 (d1) and shock2 (d2)
- **deposits**: Sum of deposits D of banks (of their balance $L+C+R = D+E$)
- **equity**: Sum of equity E of all banks: $L + C + R = D + E$ (after repayments)
- **fitness**: Fitness (μ) of the bank
- **gcs**: When we use an Erdös–Rényi graph, the Giant Component Size is the largest number of nodes that are interconnected.
- **grade_avg**: Average number of edges (connections) for the total banks
- **ir**: Interest rate r of the bank who are potential lenders ($\Delta D > 0$)
- **ir_avg**: Average interest rate r of the bank who have really loans with borrowers
- **liquidity**: Total liquidity L of the Banks $L + C + R = D + E$
- **loans**: Total amount borrowed by the banks
- **num_loans**: Num of loans. Value for **stats_market** and normal one will be the same
- **num_of_rationed**: Number of banks that were rationed in this step (needed money and were without any possible lender)
- **potential_lenders**: Number of banks in the first shock having a positive shock (ΔD)
- **prob_bankruptcy**: Probability of bankruptcy $p_b = 1 - \frac{E}{E_{max}}$, between [0..1]
- **profits**: Profits obtained in that step
- **psi**: Power market (psi) of the banks who are potential lenders, value [0..1]

- **rationing**: Total amount of the loans l of the banks
- **reserves**: Reserves R in the balance $L + C + R = D + E$
- **var_D**: Sum of $\Delta D1$ and $\Delta D2$
- **var_D1**: Amount of (ΔD) for the bank in first shock
- **var_D2**: Amount of (ΔD) for the bank in second shock

The different statistics of information obtained in table 1 are classified as:

- Global: using –**save filename**: each data column in **filename.gdt** will be obtained for all the N banks in the model for all instants time T (rows)
- With –**stats_market** what we obtain will be a file as **filenameb.gdt** for the subsets of banks that in each time are engaged in a real loan. So if in the time t there are no loans, it is removed from this statistics. The special value `real_t` indicates which was the original time.
- Individual is data obtained when we use –**detail_times** or –**detail_banks**, and it stores **filename_detailed.gdt** of those moments for all the banks individually or specific banks.
- Graphs are data obtained also in **filename.gdt**, but only we have a **LenderChange** algorithm with a random graph.

Name	Type	Global	**stats_market**	Individual	Graphs
asset_i	float	$\overline{x}/0$	$\overline{x}/0$	✓	
asset_j	float	$\overline{x}/0$	$\overline{x}/0$	✓	
bad_debt	float	$\sum$	$\sum$	✓	
bankruptcies	integer	$\sum$	$\sum$	✓	
capacity	float	$\overline{x}/nan$	$\overline{x}/nan$		
communities	integer				✓
communities_not_alone	integer				✓
d1,d2	float	$\sum$	$\sum$		
deposits	float	$\sum$	$\sum$	✓	
equity	float	$\sum$	$\sum$		
fitness	float	$\overline{x}$	$\overline{x}/nan$	✓	
gcs	integer				✓
grade_avg	integer				✓
ir,ir_avg	float	$\overline{x}/0$	$\overline{x}/nan$	✓	
liquidity	float	$\sum$	$\sum$	✓	
loans	float	$\sum$	$\sum$	✓	
num_loans	integer	✓	✓	✓	
num_of_rationed	integer	✓	✓		
prob_bankruptcy	float	$\overline{x}/nan$	$\overline{x}/nan$	✓	
profits	float	$\sum$	$\sum$		✓
psi	float	$\overline{x}/0$	$\overline{x}/nan$		✓
rationing	float	$\sum$	$\sum$		✓
reserves	float	$\sum$	$\sum$		✓
var_D,var_D1,var_D2	float	$\sum$	$\sum$		✓

Table 1: Legend for the different columns are: ✓=value without any modification. $\sum$=sum of the value for all banks. $\overline{x}$ = average of the value for all banks. 0 = No banks in this statistic. *nan*=Instead of zero, the value of "not a number" is used

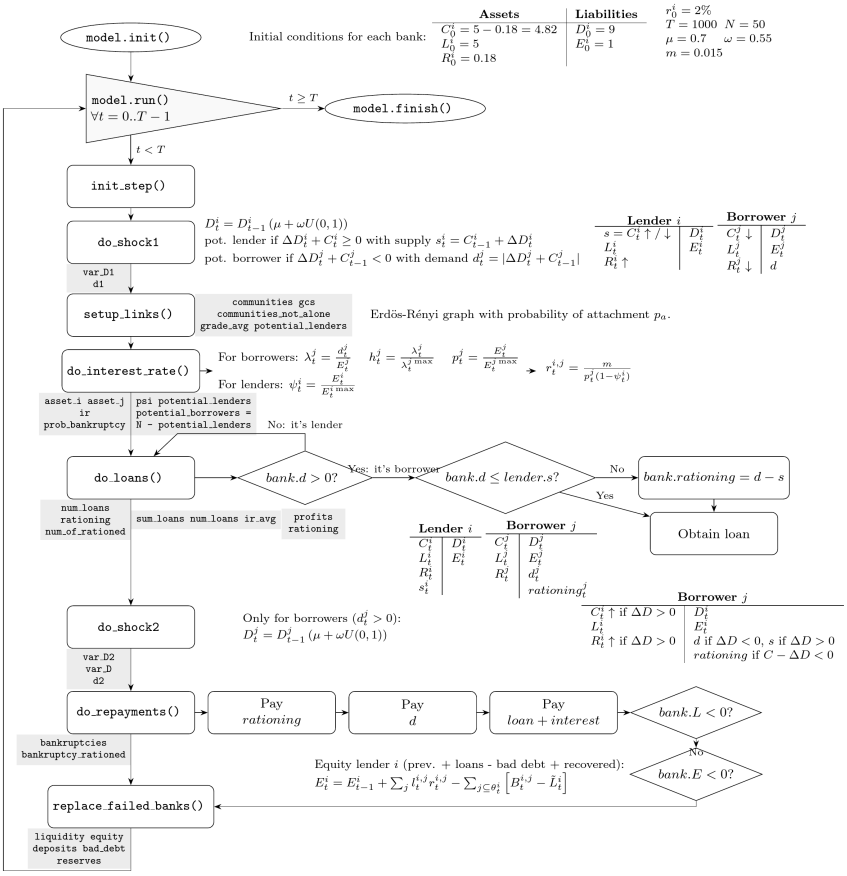

Figure 1: Sequence of steps: grey boxes indicates moments in which that statistic is obtained